HIỂU CODE
QUẢN LÝ THIẾT BỊ
PHÒNG HỌC

Lộ trình dựng dự án • Giải thích source • Thực hành • 40 câu vấn đáp

Dành cho Ma Đức Duy

Đối chiếu source nhánh master, commit 3cfa383bd16d27ebd2ac768c22ba30846c4b7bcc.

Cách dùng: mở tài liệu cạnh Visual Studio; học một phần, chỉ đúng hàm, chạy thử rồi tự giải thích.

MỤC LỤC

1. Cách học để thật sự hiểu **6**

Lộ trình bảy buổi 6

2. Bài toán trong một phút **7**

Mẫu trình bày mở đầu 7

3. Bản đồ source: mở file nào để xem việc gì? **8**

Vì sao có bốn project nhưng gọi là ba tầng? 8

4. Những cú pháp C# phải đọc được **9**

5. Dựng lại dự án theo thứ tự hợp lý **10**

Bước A — Tạo solution và dependency 10

Bước B — Mô hình trước, giao diện sau 10

Bước C — Truy cập SQL 10

Bước D — Nghiệp vụ 10

Bước E — Giao diện và sự kiện 10

Bước F — Kiểm tra toàn bộ 10

6. Entity và quan hệ một–nhiều **11**

Đọc annotation qua entity thật 11

7. DbContext, Code First, Migration và Seed **13**

Context làm gì? 13

Code First không phải Database First 13

Seed khác Migration ở đâu? 13

8. DTO và interface: cầu nối các tầng **14**

9. Nhập mới: truy vết từng chặng **15**

Sơ đồ cần thuộc và giải thích được 15

Code nghiệp vụ ngắn nhưng làm đúng một trách nhiệm 15

Phân tích kiểm tra đầu vào 15

DAL lưu như thế nào? 15

10. Sửa và Xóa: khác Nhập ở điểm nào? **17**

Sửa bảo vệ khóa chính ở ba nơi 17

Xóa phải xác nhận trước khi gọi nghiệp vụ . . . 17

11. Data binding và trạng thái Form **18**

Binding hai chỗ dễ bị hỏi . . . 18

Bảng trạng thái . . . 18

12. Khởi động, async/await và xử lý lỗi **19**

Từ Main tới dữ liệu trên màn hình . . . 19

Vì sao cần Task.Run? . . . 19

Ba kết quả phải phân biệt . . . 19

13. Cấu hình kết nối và những lỗi từng gặp **20**

14. Phần Thiết bị mạng trong bản hiện tại **21**

15. Kiểm thử và các giới hạn phải nói đúng **22**

Bộ tình huống tự chạy trên máy . . . 22

16. Bài tập sửa code để gặp câu lạ vẫn làm được **23**

Bài 1 — Thêm loại thiết bị mới . . . 23

Bài 2 — Số lượng tối đa 1000 . . . 23

Bài 3 — Thêm vị trí phòng cho thiết bị . . . 23

Bài 4 — Tìm theo tên thiết bị . . . 23

Bài 5 — Đổi giá hoặc thêm loại có phải sửa database schema? . . . 23

17. Ngân hàng 40 câu vấn đáp **24**

Nhóm A — Kiến trúc . . . 24

Nhóm B — Dữ liệu và EF . . . 24

Nhóm C — Nghiệp vụ . . . 25

Nhóm D — GUI và async . . . 25

Nhóm E — Mở rộng và kiểm thử . . . 26

18. Vấn đáp giả lập và tờ ôn nhanh **27**

Kịch bản 1 — “Em chỉ một lần Nhập chạy qua đâu?” . . . 27

Kịch bản 2 — “Không cho sửa mã, em làm ở đâu?” . . . 27

Kịch bản 3 — “Thêm MANG nhưng Wi-Fi vẫn là máy tính?” . . . 27

Kịch bản 4 — “Em thêm cột phòng học ngay bây giờ?” . . . 27

Ôn 15 phút trước khi thi . . . 27

Thang tự đánh giá . . . 27

Phụ lục. Chỉ mục hàm trong bản source được đối chiếu **28**

Dành cho: Ma Đức Duy. Công nghệ: C#, Windows Forms, .NET Framework 4.8, Entity Framework 6.5.1, SQL Server/LocalDB.

Tài liệu đối chiếu source trong repo maduygg-coder/QuanLyThietBi-midterm, nhánh master, commit 3cfa383bd16d27ebd2ac768c22ba30846c4b7bcc. Nếu em sửa code sau mốc này, cần đối chiếu lại tên hàm và hành vi. Không có tài liệu nào bảo đảm trả lời mọi câu hỏi; mục tiêu là hiểu nguyên lý, chỉ đúng code và suy luận được khi câu hỏi thay đổi.

1. Cách học để thật sự hiểu

Mỗi phần học theo bốn bước: đọc lời giải thích → mở đúng file → chạy một tình huống → đóng tài liệu và tự nói lại. Chưa giải thích được bằng lời đơn giản thì chưa chuyển sang học thuộc câu trả lời.

Ưu tiên bắt buộc: kiến trúc ba tầng; quan hệ một–nhiều; EF Code First/Migration; kiểm tra dữ liệu trong BLL; luồng Nhập/Sửa/Xóa; khóa chính khi sửa; trạng thái giao diện. Sau đó học async/await, cấu hình kết nối, kiểm thử và mở rộng.

Lộ trình bảy buổi

Buổi	Nội dung	Sản phẩm tự kiểm tra
1, 45–60 phút	Bài toán, bảng dữ liệu, bản đồ source	Vẽ được 2 bảng và quan hệ 1–n
2, 60 phút	Dựng solution, entity, context, migration	Giải thích được mỗi project và annotation
3, 60–75 phút	BLL và kiểm tra nhập liệu	Truy vết được 5 dữ liệu sai dừng ở đâu
4, 60 phút	Nhập, Sửa, Xóa từ GUI tới SQL	Kể tên đúng hàm ở cả ba tầng
5, 45–60 phút	Binding, cờ trạng thái, async, lỗi	Giải thích được nút bật/tắt và Form không bị treo
6, 60 phút	Thay đổi yêu cầu, kiểm thử, Git	Tự thêm một quy tắc và kiểm tra biên
7, 45–60 phút	Vấn đáp thử và demo	Trả lời ngắn rồi mở code làm bằng chứng

Không cần học hết một lần. Sau mỗi buổi, ghi ba điều hiểu được và một điều còn vướng. Phần trả lời ở cuối dùng để đối chiếu sau khi đã thử trả lời.

2. Bài toán trong một phút

Ứng dụng quản lý thiết bị phòng học theo loại. Người dùng nhập mã, tên, ngày mua, số lượng, đơn giá và chọn loại. Ứng dụng cho xem danh sách, nhập mới, sửa, xóa; dữ liệu được lưu trong SQL Server để đóng/mở lại vẫn còn.

Một dòng TB001 là một bản ghi thiết bị. Nếu SoLuong=10, nghĩa là bản ghi đó quản lý số lượng 10, không phải tự tạo 10 dòng. MaTB phân biệt các bản ghi; MaLoai gắn bản ghi với loại thiết bị.

Bản hiện tại có bốn loại mẫu: Thiết bị trình chiếu, Thiết bị máy tính, Thiết bị âm thanh, Thiết bị mạng. Thiết bị mẫu chỉ được thêm khi database hoàn toàn trống từ đầu. Loại mẫu còn thiếu được bổ sung theo mã khi Seed chạy.

Mẫu trình bày mở đầu

“Ứng dụng của em quản lý thiết bị phòng học theo loại. Em tách giao diện WinForms, nghiệp vụ BLL và truy cập SQL bằng EF ở DAL. Hai entity là LoaiThietBi và ThietBi có quan hệ một–nhiều. Khi nhấn Nhập, GUI lấy dữ liệu, BLL kiểm tra và chặn mã trùng, DAL lưu bằng SaveChanges; sau đó GUI tải lại danh sách.”

Hãy hiểu và diễn đạt lại, không cần đọc nguyên văn. Chỉ nói những chức năng thật sự có trong bản đang trình bày; bài thiết bị này chưa có chức năng tìm kiếm như bản sách trước đó.

3. Bản đồ source: mở file nào để xem việc gì?

Nơi mở trong Solution Explorer	Vai trò
GUI/Program.cs	Điểm vào Main, nối service/repository, chạy Form
GUI/FrmThietBi.cs	Tạo giao diện, trạng thái nút, lấy dữ liệu, xử lý thao tác
GUI/FrmDatabaseSettings.cs	Hộp cấu hình máy chủ và database
GUI/App.config	Tên sinh viên, connection string, provider EF
BLL/ThietBiService.cs	Kiểm tra dữ liệu và nghiệp vụ Nhập/Sửa/Xóa
BLL/DatabaseSetupService.cs	Kiểm tra cấu hình trước khi thử/lưu
Entities/ThietBi.cs, LoaiThietBi.cs	Entity, khóa, độ dài, quan hệ
Entities/Contracts.cs	DTO Input/Row và interface repository
Entities/DatabaseConnectionOptions.cs	DTO cấu hình và hợp đồng lưu cấu hình
DAL/Repositories.cs	Truy vấn, thêm, sửa, xóa thực tế qua EF
DAL/ThietBiContext.cs	DbContext, DbSet và cấu hình model
DAL/DatabaseBootstrapper.cs	Kiểm tra máy chủ, áp dụng Migration
DAL/ConnectionSettingsStore.cs	Đọc/lưu cấu hình, thử kết nối SQL
DAL/Migrations/Configuration.cs	Cấu hình migration và Seed
DAL/Migrations/*_InitialCreate.cs	Up/Down tạo hoặc gỡ schema
Tests/Program.cs	Kiểm tra BLL với repository giả

Các đường dẫn rút gọn GUI/BLL/DAL/Entities tương ứng với project QuanLyThietBi.GUI/BLL/DAL/Entities.

Vì sao có bốn project nhưng gọi là ba tầng?

Ba tầng chức năng là GUI, BLL, DAL. Entities là phần mô hình/hợp đồng dùng chung, không phải một tầng chức năng thứ tư. Project là đơn vị tổ chức/build; tầng là cách chia trách nhiệm. Hai khái niệm không đồng nhất.

Luồng thao tác nghiệp vụ:

```
Người dùng → FrmThietBi → ThietBiService → repository DAL → SQL Server
                ↑ k□t qu□ hoặc l□i được tr□ v□ qua các lời gọi ↑
```

BLL chỉ tham chiếu Entities và gọi interface. DAL cũng tham chiếu Entities nhưng triển khai interface bằng EF. Program trong GUI tạo repository cụ thể rồi truyền cho service; đó là chỗ nối dependency, không phải Form tự truy vấn database. Luồng khởi tạo schema riêng được Program truyền vào Form bằng delegate DatabaseBootstrapper.Initialize.

4. Những cú pháp C# phải đọc được

Cú pháp trong bài	Hiểu đơn giản
class, property get; set;	Class mô tả kiểu đối tượng; property giữ/đọc/ghi giá trị
new ThietBi { ... }	Tạo đối tượng và gán thuộc tính; chưa tự lưu vào SQL
public / private	Gọi được từ bên ngoài class / chỉ dùng trong class
readonly	Không gán lại field sau khi constructor hoàn tất; không có nghĩa đối tượng bên trong bất biến
sealed	Không cho class khác kế thừa class này
interface	Hợp đồng các phương thức phải có, không phải dữ liệu đang nằm trong SQL
List<T>, ICollection<T>	Danh sách / hợp đồng tập hợp các phần tử kiểu T
using (var db=...)	Bảo đảm Dispose tài nguyên khi rời khối, kể cả có exception
using System.Linq;	Nhập namespace; khác với khối using quản lý tài nguyên
input.MaTB ?? ""	Nếu MaTB là null thì dùng chuỗi rỗng
Trim()	Bỏ khoảng trắng đầu/cuối; không bỏ mọi khoảng trắng ở giữa
x => x.MaTB == ma	Lambda, ở đây là điều kiện nhận x và so mã
=> ở một phương thức	Cách viết gọn phương thức một biểu thức
nameof(MaLoai)	Tạo tên thuộc tính dạng chuỗi và được compiler kiểm tra
throw new ...	Phát sinh lỗi để dừng luồng hiện tại và chuyển tới catch phù hợp
try / catch / finally	Thử thao tác / xử lý lỗi / dọn trạng thái khi kết thúc
async Task, await	Công việc có thể chờ bất đồng bộ; await không có nghĩa bỏ qua lỗi
Action, Func<DateTime>	Delegate không trả giá trị / delegate trả DateTime
[STAThread]	Đặt mô hình apartment cho luồng giao diện Windows theo yêu cầu tương tác COM

Bài tập: tìm mỗi cú pháp một chỗ trong source và giải thích bằng dữ liệu TB006. Chỉ nhớ tên cú pháp mà không biết nó đang làm gì trong bài thì chưa đủ.

5. Dựng lại dự án theo thứ tự hợp lý

Đây là trình tự để em tự dựng lại, không phải khẳng định mọi file trong lịch sử được viết đúng thứ tự này.

Bước A — Tạo solution và dependency

Tạo Windows Forms .NET Framework 4.8 cho GUI, ba Class Library net48 cho BLL, DAL, Entities. Thêm reference: BLL → Entities; DAL → Entities; GUI → BLL, DAL, Entities. Cài EntityFramework 6.5.1 cho DAL và GUI để build/copy provider khi chạy. Đặt GUI là Startup Project.

Dự án đang dùng csproj SDK-style với TargetFramework=net48 và reference WinForms tường minh. Đây vẫn là .NET Framework 4.8, không phải GUI chạy bằng .NET 8. Project Tests là console net8 riêng và không nằm trong solution bốn project chính.

Bước B — Mô hình trước, giao diện sau

Viết LoaiThietBi và ThietBi, xác định PK/FK, độ dài và navigation. Viết DTO/interface để BLL và DAL thống nhất cách trao đổi dữ liệu. Không đưa TextBox hoặc MessageBox vào Entities/DAL.

Bước C — Truy cập SQL

Tạo ThietBiContext, hai DbSet, cấu hình decimal và quan hệ. Đặt chuỗi kết nối trong App.config. Tạo Migration cho schema; cấu hình Seed kiểm tra tồn tại trước khi thêm. Kiểm tra SQL chạy trước khi đánh giá chức năng nhập liệu.

Bước D — Nghiệp vụ

Viết KiemTra, Nhap, Sua, Xoa trong ThietBiService. Test đầu vào sai trước để chắc nó không đi tới lưu dữ liệu. BLL không gọi trực tiếp DbContext.

Bước E — Giao diện và sự kiện

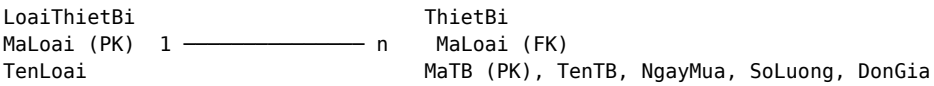
Tạo control, cột lưới, TabIndex, AcceptButton và CancelButton. Nối nút với hàm, rồi xây trạng thái nhập mới/chọn dòng/đang xử lý. Sau cùng thêm cấu hình kết nối, progress và xử lý lỗi.

Bước F — Kiểm tra toàn bộ

Chạy SQL thật trên Windows: nhập TB006 → sửa → đóng/mở lại → xóa → đóng/mở lại. Thử dữ liệu sai, mã trùng, hủy xóa và hủy thoát. Lưu source bằng commit có mô tả phần thật sự thay đổi.

6. Entity và quan hệ một–nhiều

Bảng	Cột	Kiểu và ràng buộc
LoaiThietBi	MaLoai	string, khóa chính, tối đa 10
LoaiThietBi	TenLoai	string bắt buộc, tối đa 100
ThietBi	MaTB	string, khóa chính, tối đa 20
ThietBi	TenTB	string bắt buộc, tối đa 150
ThietBi	NgayMua	DateTime, lưu SQL datetime
ThietBi	SoLuong	int; nghiệp vụ yêu cầu > 0
ThietBi	DonGia	decimal(18,2); nghiệp vụ yêu cầu > 0
ThietBi	MaLoai	string FK bắt buộc, tối đa 10



Một loại MANG có thể có nhiều thiết bị: router, switch, access point. Một bản ghi thiết bị có đúng một MaLoai bắt buộc. Tên loại không phải khóa ngoại; hai loại có thể có tên giống nhau nếu không đặt ràng buộc unique riêng.

Đọc annotation qua entity thật

```
using System;
using System.ComponentModel.DataAnnotations;
using System.ComponentModel.DataAnnotations.Schema;
namespace QuanLyThietBi.Entities
{
    [Table("ThietBi")]
    public class ThietBi
    {
        [Key, Required, StringLength(20)]
        public string MaTB { get; set; }
        [Required, StringLength(150)]
        public string TenTB { get; set; }
        public DateTime NgayMua { get; set; }
        public int SoLuong { get; set; }
        [Column(TypeName = "decimal")]
        public decimal DonGia { get; set; }
        [Required, StringLength(10)]
        public string MaLoai { get; set; }
        [ForeignKey(nameof(MaLoai))]
        public virtual LoaiThietBi LoaiThietBi { get; set; }
    }
}
```

Key xác định khóa chính. StringLength đặt độ dài. Required trên chuỗi khai báo bắt buộc trong model; BLL vẫn kiểm tra chuỗi trắng và trim trước khi lưu. ForeignKey đặt trên navigation để chỉ ra property MaLoai là FK.

Navigation ThietBi.LoaiThietBi giúp đi từ thiết bị sang loại; LoaiThietBi.ThietBis là chiều ngược. virtual có thể phục vụ proxy/lazy loading khi cấu hình cho phép, nhưng bài này tắt LazyLoadingEnabled và ProxyCreationEnabled. Không được nói “cứ virtual là luôn tự tải loại”.

Lưu ý quan trọng: int và decimal không nullable không đồng nghĩa phải lớn hơn 0. Required không thay thế quy tắc số lượng/giá dương. Trong bài hiện tại, các quy tắc này nằm ở BLL; chưa có CHECK constraint riêng trong SQL để chặn mọi đường ghi ngoài ứng dụng.

7. DbContext, Code First, Migration và Seed

Context làm gì?

ThietBiContext kế thừa DbContext. Hai DbSet dùng để truy vấn/quản lý entity. Constructor đọc connection string qua ConnectionSettingsStore, tạo SqlConnection do DAL quản lý và tắt lazy loading/proxy. DbContext được Dispose sau mỗi thao tác repository.

```
protected override void OnModelCreating(DbModelBuilder modelBuilder)
{

    modelBuilder.Entity<ThietBi>().Property(x => x.DonGia).HasPrecision(18, 2);
    modelBuilder.Entity<ThietBi>().HasRequired(x => x.LoaiThietBi)
        .WithMany(x => x.ThietBis).HasForeignKey(x => x.MaLoai).WillCascadeOnDelete(false);
    base.OnModelCreating(modelBuilder);
}
```

HasPrecision(18,2) nghĩa là 18 chữ số tổng cộng, 2 sau dấu thập phân: tối đa 16 chữ số phần nguyên. EF6 không có Precision attribute như một số phiên bản EF Core mới, nên bài dùng Column annotation kết hợp Fluent API và migration rõ precision/scale.

HasRequired → mỗi thiết bị phải có loại. WithMany → một loại nhiều thiết bị. HasForeignKey → dùng MaLoai để liên kết. WillCascadeOnDelete(false) → SQL không tự xóa hết thiết bị khi xóa một loại; cần xử lý thiết bị phụ thuộc trước nếu mở rộng chức năng xóa loại.

Code First không phải Database First

Code First lấy class C# và cấu hình model làm nguồn, dùng Migration tạo/chuyển đổi schema. Database First thường xuất phát từ database đã có để sinh model. Snapshot model XML trong docs và metadata nén của Migration là phần mô tả model Code First; không phải bằng chứng dùng Database First.

Trong Up của InitialCreate: tạo LoaiThietBi trước, tạo ThietBi rồi FK/index. Down gỡ FK/index trước, xóa bảng con rồi bảng cha. __MigrationHistory giúp EF xác định những migration nào đã áp dụng; không dùng nó để đo người làm bài đã viết code bao lâu.

```
Add-Migration TenThayDoi -ProjectName QuanLyThietBi.DAL -StartUpProjectName QuanLyThietBi.GUI
Update-Database -ProjectName QuanLyThietBi.DAL -StartUpProjectName QuanLyThietBi.GUI
```

Không chạy Add-Migration chỉ vì đổi nhãn cột, tên sinh viên hoặc bổ sung một loại dữ liệu. Những việc đó không thay schema. Đổi entity có thêm cột lưu SQL thì mới cần migration tương ứng.

Seed khác Migration ở đâu?

Migration đổi cấu trúc bảng; Seed khởi tạo dữ liệu. Configuration.cs tính taoMau khi cả hai bảng trống. Trong transaction: thêm loại còn thiếu, SaveChanges; nếu taoMau thì thêm 5 thiết bị và SaveChanges; sau đó Commit. Nếu lỗi trước Commit, transaction bị Dispose/rollback, tránh chỉ có một phần dữ liệu mẫu.

Seed không dùng để reset dữ liệu mỗi lần mở. Thiết bị đã sửa/xóa không bị tạo lại vì database vẫn có các loại. Có nhiều SaveChanges trong cùng transaction vẫn có thể được commit/rollback như một đơn vị.

8. DTO và interface: cầu nối các tầng

ThietBiInput giữ MaTB/TenTB/MaLoai, ngày và hai ô số dạng string. Lý do: BLL phải nhận được cả “abc” hoặc “1,5” để tự xác định lỗi định dạng. Sau kiểm tra, BLL tạo entity có SoLuong int, DonGia decimal.

ThietBiRow là dữ liệu phục vụ lưới, có thêm TenLoai. Nó không phải bảng mới chỉ vì được khai báo class. Interface IThietBiRepository định nghĩa LayTatCa, TonTai, Them, Sua, Xoa. BLL không cần biết những hàm đó được EF hay repository giả thực hiện.

Ở Program.cs, service nhận ThietBiRepository và LoaiThietBiRepository qua constructor. Đây là constructor dependency injection được nối thủ công, không dùng container bên ngoài. Lợi ích là test có thể truyền MemoryThietBi, không cần SQL Server.

9. Nhập mới: truy vết từng chặng

Sơ đồ cần thuộc và giải thích được

```
btnNhap.Click
→ LuuAsync(false)
→ DocDuLieu() tạo ThietBiInput
→ ThucHienGhiAsync(...)
→ BLL.Nhap(input)
→ KiemTra(input)
→ DAL.TonTai(ma)
→ DAL.Them(entity): Add + SaveChanges
→ LayDanhSach()
→ GanDanhSach() + LamMoi() + thông báo
```

Code nghiệp vụ ngắn nhưng làm đúng một trách nhiệm

```
public void Nhap(ThietBiInput input)
{
    var entity = KiemTra(input);
    if (thietBi.TonTai(entity.MaTB)) throw new BusinessException("Mã thi□t bị đã t□n tại.");
    thietBi.Them(entity);
}
```

KiemTra có thể throw BusinessException; khi đó Nhap dừng và không gọi Them. TonTai kiểm tra trước để báo thân thiện. Vẫn phải có PK ở database vì hai chương trình có thể cùng kiểm tra “chưa tồn tại” rồi cùng thêm. Một phiên sẽ bị SQL từ chối; không có bảo đảm tránh race chỉ bằng if ở BLL.

Phân tích kiểm tra đầu vào

```
var ma = (input.MaTB ?? "").Trim();
```

Nếu MaTB=null thì dùng chuỗi rỗng để không NullReference khi gọi Trim. “ TB006 ” trở thành “TB006”. Mã toàn khoảng trắng trở thành rỗng và bị báo lỗi.

```
if (input.NgayMua.Date > ngayHienTai().Date)
    throw new BusinessException("Ngày mua không được lớn hơn ngày hiện tại.");
```

So theo ngày, không theo giờ. Func<DateTime> cho phép test cố định hôm nay; khi chạy thật mặc định là DateTime.Today. Giới hạn 01/01/1753 được thêm vì dùng SQL datetime, không phải mọi DateTime của C# đều lưu được vào kiểu này.

TryParse số lượng trả false nếu nhập chữ, số phân số hoặc vượt Int32; điều kiện <=0 chặn 0 và âm. Giá dùng culture vi-VN và chỉ cho dấu thập phân/phần dấu: 15000000,50 hợp lệ; không tự coi “15.000.000” là số tiền hợp lệ trong ô nhập.

Giá còn bị chặn vượt decimal(18,2) và có phần thập phân không biểu diễn chính xác trong 2 chữ số. 1,001 bị từ chối; 1,230 có cùng giá trị số với 1,23 nên không nhất thiết bị từ chối. Quy tắc đang kiểm tra giá trị, không đơn thuần đếm ký tự sau dấu phẩy.

DAL lưu như thế nào?

```
public void Them(ThietBi entity)
{
    using (var db = new ThietBiContext()) { db.ThietBis.Add(entity); db.SaveChanges(); }
}
```

Add đưa entity vào trạng thái Added của context; chưa có nghĩa SQL đã có dòng. SaveChanges mới gửi thay đổi xuống database. using bảo đảm Dispose kể cả khi lưu phát sinh exception. Việc tạo context mỗi thao tác không có nghĩa luôn tạo kết nối vật lý mới vì SqlClient còn có connection

pooling.

Bài tập: điền TB006, tên Router, loại MANG, hôm nay, số lượng 2, giá 1200000. Theo dấu tới SaveChanges. Sau đó thử lại mã “ TB006 ” và chứng minh nó không tới Them.

10. Sửa và Xóa: khác Nhập ở điểm nào?

Sửa bảo vệ khóa chính ở ba nơi

ChonDong lấy ThietBiRow, lưu maDangChon, đưa dữ liệu vào ô nhập, chọn loại bằng MaLoai và đặt txtMaTB.ReadOnly=true. BLL.Sua so mã gốc với mã sau chuẩn hóa. DAL.Sua tìm bản ghi theo maGoc và chỉ gán các thuộc tính không khóa.

```
public void Sua(string maGoc, ThietBi entity)
{
    using (var db = new ThietBiContext())
    {
        var current = db.ThietBis.Find(maGoc);
        if (current == null) throw new InvalidOperationException("Thi□t bị đã bị xóa. Hãy t□i
    lại danh sách.");
        current.TenTB=entity.TenTB; current.NgayMua=entity.NgayMua;
        current.SoLuong=entity.SoLuong; current.DonGia=entity.DonGia;
    current.MaLoai=entity.MaLoai;
        db.SaveChanges();
    }
}
```

Find tìm theo PK, có thể trả null nếu bản ghi đã bị phiên khác xóa. Entity trả về đang được context theo dõi. Gán thuộc tính rồi SaveChanges là đủ; không cần Add và không cần tạo dòng mới. Không gán current.MaTB là chủ ý bảo vệ PK.

Bài tập: đổi số lượng TB006 từ 2 lên 7. Giải thích tại sao tổng số dòng trong bảng không tăng. Nếu người khác xóa TB006 trước khi em lưu, repository phải báo lỗi, không được im lặng báo thành công.

Xóa phải xác nhận trước khi gọi nghiệp vụ

XoaAsync kiểm tra có dòng đang chọn, lấy mã vào biến cục bộ rồi hiện Yes/No. Nếu không Yes thì return trước khi gọi service. BLL kiểm tra mã/tồn tại; DAL Find, Remove và SaveChanges. Tồn tại có thể thay đổi giữa BLL và DAL nên DAL vẫn kiểm tra null.

Bài tập: chọn No trong hộp xóa, kiểm tra dòng còn nguyên. Chọn Yes, đóng/mở lại ứng dụng, kiểm tra dữ liệu thật đã mất. Không nhầm xóa dòng khỏi DataGridView với xóa khỏi SQL.

11. Data binding và trạng thái Form

Binding hai chỗ dễ bị hỏi

ComboBox dùng DisplayMember=TenLoai, ValueMember=MaLoai. Người dùng thấy tên nhưng chương trình lưu mã. Khi chọn TB003, gán SelectedValue=row.MaLoai để chọn đúng loại; không chọn bằng vị trí hàng hoặc dựa vào chuỗi tên.

DataGridView có AutoGenerateColumns=false và các DataPropertyName tương ứng ThietBiRow. DAL chiếu TenLoai=x.LoaiThietBi.TenLoai nên cột loại hiện tên. LINQ to Entities có thể dịch truy cập navigation trong phép chiếu thành truy vấn SQL phù hợp ngay cả khi lazy loading tắt; không cần dựa vào tải lười để lấy tên.

Bảng trạng thái

Trạng thái	Nhập	Sửa/Xóa
Chưa kết nối / nạp lỗi	Tắt	Tắt
Mới, đã sẵn sàng	Bật	Tắt
Chọn dòng	Tắt	Bật
Đang nạp hoặc ghi	Tắt	Tắt

Bốn biến quan trọng: sanSang cho biết có dữ liệu/kết nối để thao tác; dangXuLy chặn lệnh mới khi đang xử lý; maDangChon phân biệt nhập và sửa; dangGanDuLieu bỏ qua SelectionChanged do chương trình bind/reset, thay vì do người dùng chọn.

LamMoi xóa nội dung, ngày hôm nay, SelectedIndex=-1, bỏ chọn dòng và mở khóa mã. CapNhatNut tính trạng thái nút từ các biến thay vì rải các lệnh bật/tắt không nhất quán. Chọn dòng không hợp lệ phải return.

Enter được gắn với btnNhap bằng AcceptButton. Khi chọn dòng, btnNhap tắt, Enter không tự thêm. Esc gắn với nút Thoát; nút gọi Close, còn một FormClosing duy nhất hỏi xác nhận. Chọn Không đặt e.Cancel=true. Không hỏi thêm ở btnThoat.Click vì sẽ dễ hiện hai hộp thoại.

12. Khởi động, async/await và xử lý lỗi

Từ Main tới dữ liệu trên màn hình

Main thiết lập culture vi-VN, giao diện Windows, tạo service và settings rồi Application.Run(new FrmThietBi(...)). Constructor Form tạo control/nối sự kiện, chưa truy vấn SQL ngay. Sự kiện Shown gọi NapDuLieuAsync để cửa sổ hiện trước.

Trong tác vụ nền: kiểm tra SQL ở master → DbMigrator.Update tạo/cập nhật database và seed → lấy danh sách loại và thiết bị. Sau await, UI thread bind control. Lúc load chờ SQL không nên coi là thiếu chức năng chỉ vì lưới chưa có dữ liệu.

Vì sao cần Task.Run?

Repository đang dùng EF đồng bộ. Task.Run chuyển công việc chặn này khỏi UI thread để cửa sổ vẫn xử lý vẽ/sự kiện. Sau await bắt đầu từ UI, continuation thông thường trở lại WinForms synchronization context. Không truy cập TextBox/ComboBox trực tiếp trong Task.Run; DocDuLieu chụp input trên UI trước.

Task.Run không khiến SQL nhanh hơn và không tự tạo database thành công. DbContext không thread-safe: bài tạo context cục bộ ở mỗi repository, không chia sẻ một context cho nhiều tác vụ. Các IsDisposed/Disposing giúp không cập nhật control sau khi cửa sổ đã đóng; không có nghĩa tác vụ SQL đang chạy đã được hủy tự động.

Ba kết quả phải phân biệt

- BusinessException: sai nghiệp vụ, hiện cảnh báo và giữ nội dung để sửa.

- Ghi chưa hoàn tất: hiện lỗi, không báo thành công. Khi mạng đứt đúng lúc commit, trạng thái có thể chưa xác định; nên tải lại trước khi làm lại.

- daLuu=true nhưng LayDanhSach lỗi: thông báo đã lưu nhưng chưa tải lại được, khóa ghi cho tới khi nạp lại; tránh người dùng nhập lại và tạo trùng.

finally dọn dangXuLy, dừng timer và cập nhật nút. Timer chỉ hiển thị thời gian chờ; không phải cơ chế timeout hoặc cơ chế hủy SQL.

13. Cấu hình kết nối và những lỗi từng gặp

App.config có ThietBiDb: máy chủ, Initial Catalog, Integrated Security và provider System.Data.SqlClient. Integrated Security=True dùng tài khoản Windows, không phải tài khoản GitHub. Tên sinh viên nằm ở appSettings/HoTenSinhVien, không liên quan đến bảng thiết bị.

FrmDatabaseSettings → DatabaseSetupService.TestAndSave → ConnectionSettingsStore.TestServer/Save. BLL kiểm tra chuỗi rỗng và độ dài; DAL thử kết nối master vì database bài làm có thể chưa tồn tại. Kết nối master thành công chưa chứng minh có quyền tạo database hoặc toàn bộ migration sẽ thành công.

Cấu hình runtime được lưu cạnh EXE trong QuanLyThietBi.GUI.exe.config. Build có thể chép App.config nguồn đè file đó. DAL đọc cấu hình hiện tại khi tạo context để nhận máy chủ/database vừa đổi, tránh chỉ dựa vào cấu hình EF đã cache.

Hiện tượng	Cần kiểm tra trước
Các project incompatible	Workload .NET desktop development và targeting pack; restart/reload
NETSDK1004	Restore NuGet, mở solution thay vì chỉ Folder View
Class Library cannot be started	Đặt QuanLyThietBi.GUI làm Startup Project
SQL cannot find file / không kết nối	LocalDB đã cài/chạy chưa, tên instance có đúng không
Lưới và loại đều trống, nút bị mờ	Dòng trạng thái/hộp lỗi: đang nạp hay nạp thất bại
Login/CREATE DATABASE denied	Tài khoản và quyền SQL; không chỉ sửa TextBox
Sửa tên mà không thấy đổi	Sửa đúng App.config của GUI đang build, lưu rồi chạy lại
Clone xong chạy không được	Open QuanLyThietBi.sln, Restore, chọn GUI startup

LocalDB có thể kiểm tra bằng sqllocaldb info và sqllocaldb start MSSQLLocalDB. Chỉ tạo instance khi đã cài LocalDB nhưng instance chưa tồn tại. Không xóa database đang có dữ liệu để thử sửa lỗi kết nối.

14. Phần Thiết bị mạng trong bản hiện tại

Source đã thêm MaLoai=MANG, TenLoai=Thiết bị mạng trong Seed. Đây là thêm dữ liệu, không cần migration mới. Cơ chế Any trước Add ngăn thêm lại cùng mã khi chạy lại.

Điểm em phải biết khi vấn đáp: ở commit được đối chiếu, thiết bị mẫu TB003 “Bộ phát Wi-Fi TP-Link” vẫn có MaLoai=MAYTINH trong Seed. Tài liệu không sửa source hay dữ liệu hộ em. Nếu database trên máy em đã sửa TB003 sang MANG bằng nút Sửa, đó là dữ liệu runtime và không tự xuất hiện trong file C#.

Để chỉnh đúng và nhất quán: sửa MaLoai của TB003 trong dữ liệu mẫu thành MANG để lần tạo database mới đúng; với database đã có, chọn TB003 trên Form → loại Thiết bị mạng → Sửa. Không dùng Seed ghi đè tất cả thiết bị mỗi lần chạy.

Câu hỏi biến thể: “Tại sao thêm MANG rồi TB003 chưa đổi loại?” Vì danh mục loại và bản ghi thiết bị là hai dữ liệu khác nhau; Seed chỉ bổ sung loại, không tự thay FK của thiết bị hiện có.

15. Kiểm thử và các giới hạn phải nói đúng

Bộ test console dùng MemoryThietBi/MemoryLoai để gọi BLL, kiểm tra nhập hợp lệ, trùng sau Trim, ngày/số/độ dài sai, khóa bất biến, lỗi DAL và cấu hình. Func<DateTime> cố định ngày cho kết quả không phụ thuộc ngày đang chạy.

60 kiểm tra BLL là kết quả đã chạy trong giai đoạn xây bản gốc. Tài liệu này kiểm tra source hiện tại, không tuyên bố vừa chạy lại mọi test hoặc kiểm thử SQL/WinForms mới nhất. Test giả không chứng minh migration chạy được trên SQL thật; ảnh chạy được trên máy người dùng là bằng chứng khác với test tự động.

Bộ tình huống tự chạy trên máy

Đầu vào/thao tác	Mong đợi
TB006, Router, MANG, hôm nay, SL=2, giá=1200000	Lưu và tải lại được
Nhập lại “ TB006 ”	Báo trùng
Mã hoặc tên chỉ có khoảng trắng	Bị chặn ở BLL
Không chọn loại	Báo chọn loại
Ngày mai	Báo ngày không hợp lệ
SL=0, -1, abc, 1,5	Bị chặn
SL=2147483648	Vượt Int32, bị chặn
Giá=0, âm, abc, 1,001	Bị chặn
Giá=1,23	Được chấp nhận nếu các trường khác hợp lệ
Sửa TB006, cố đổi mã từ nguồn gọi khác	BLL từ chối
Xóa chọn Không	Không gọi xóa trong BLL
Đóng/mở lại sau ghi	Dữ liệu còn trong SQL
Đóng bằng X rồi chọn Không	Form ở lại, một hộp xác nhận
Máy chủ sai	Báo lỗi, không báo lưu thành công

Giới hạn hiện có: chưa có đăng nhập/phân quyền, chưa có rowversion để chống mất cập nhật khi hai người cùng sửa, chưa có CHECK constraint cho các quy tắc số dương, chưa có chức năng CRUD loại riêng. Nói thẳng giới hạn rồi trình bày hướng mở rộng là tốt hơn khẳng định ứng dụng xử lý mọi tình huống.

16. Bài tập sửa code để gặp câu lạ vẫn làm được

Bài 1 — Thêm loại thiết bị mới

Nêu file cần sửa, thêm LoaiThietBi có mã không quá 10 ký tự trong Seed. Giữ Any trước Add, không thêm bảng hay TextBox mới. Chạy lại và kiểm tra ComboBox. Giải thích vì sao không cần migration.

Bài 2 — Số lượng tối đa 1000

Thêm điều kiện ở KiemTra sau TryParse. Kiểm tra SL=1000 và 1001, sửa thông báo và test. Không chỉ chặn KeyPress ở GUI vì dữ liệu có thể tới từ nơi gọi khác.

```
if (soLuong > 1000)
    throw new BusinessException("S lượng không được vượt quá 1000.");
```

Đây là bài tập đề xuất, chưa phải quy tắc đang có trong repo. Nếu dùng thật, xem lại cả thông báo lỗi và bài test biên cũ đang chấp nhận Int32.MaxValue.

Bài 3 — Thêm vị trí phòng cho thiết bị

Cần entity có property mới, DTO Input/Row, ô nhập/cột lưới, BLL kiểm tra, DAL chiếu/ghi thuộc tính và Migration tạo cột. Nếu bảng đã có dữ liệu, phải quyết định cột nullable hay giá trị mặc định trước khi đặt NOT NULL. Không chỉ thêm một TextBox.

Bài 4 — Tìm theo tên thiết bị

Bài hiện tại chưa có. Thiết kế một chức năng tìm: GUI nhận từ khóa, BLL chuẩn hóa, DAL truy vấn hoặc lọc dữ liệu đã nạp theo quy mô. Nếu truy vấn SQL, dùng LINQ/provider để tham số hóa, không nối SQL với chuỗi người dùng. Quy định xử lý khi kết quả rỗng và khi người dùng đang sửa chưa lưu.

Bài 5 — Đổi giá hoặc thêm loại có phải sửa database schema?

Đổi giá bằng nút Sửa và thêm dòng loại là thay dữ liệu. Đổi DonGia từ decimal(18,2) sang precision/scale khác là đổi schema, phải có migration và xem xét dữ liệu hiện có.

17. Ngân hàng 40 câu vấn đáp

Cách luyện: che phần “Đáp ý chính”, nói trong 20–40 giây, sau đó mở code minh họa. Câu hỏi nâng cao có thể cần dài hơn; không áp dụng máy móc giới hạn thời gian.

Nhóm A — Kiến trúc

1. Dự án giải quyết bài toán gì? Đáp ý chính: quản lý thiết bị theo loại, CRUD, kiểm tra hợp lệ, lưu SQL qua EF.

2. Vì sao không viết hết trong Form? Đáp ý chính: tách UI, quy tắc và dữ liệu để dễ sửa/test; Form không bị phụ thuộc chi tiết SQL.

3. Vì sao bốn project mà ba tầng? Đáp ý chính: Entities dùng chung; GUI/BLL/DAL là ba trách nhiệm chức năng.

4. Service nhận repository bằng cách nào? Đáp ý chính: Program khởi tạo implementation và truyền qua constructor; phụ thuộc interface.

5. DAL có được hiện MessageBox không? Đáp ý chính: không trong thiết kế này; DAL trả kết quả/ngoại lệ để GUI quyết định hiển thị.

6. DTO có phải entity/bảng SQL không? Đáp ý chính: Input/Row phục vụ trao đổi và hiển thị; không được map thành bảng chỉ vì tồn tại class.

Nhóm B — Dữ liệu và EF

7. PK và FK khác nhau? Đáp ý chính: MaTB định danh thiết bị; MaLoai liên kết sang một loại có thật.

8. Navigation khác MaLoai thế nào? Đáp ý chính: MaLoai là giá trị FK; navigation là đối tượng/tập đối tượng biểu diễn quan hệ trong C#.

9. DbContext khác DbSet thế nào? Đáp ý chính: context quản lý phiên/model/theo dõi; DbSet là tập entity để truy vấn/ghi.

10. Add đã lưu chưa? Đáp ý chính: chưa; cần SaveChanges thành công.

11. Code First khác Database First? Đáp ý chính: từ model C# và migration so với từ database có sẵn để sinh model.

12. Up, Down và Seed khác nhau? Đáp ý chính: Up/Down đổi schema theo hai chiều, Seed thêm dữ liệu.

13. Làm sao không tạo lại bảng mỗi lần mở? Đáp ý chính: DbMigrator và __MigrationHistory theo dõi migration; tắt initializer tự tạo theo cơ chế khác.

14. Tại sao decimal(18,2)? Đáp ý chính: kiểu tiền thập phân, 18 chữ số tổng/2 số lẻ; explicit precision để SQL và BLL thống nhất.

15. Vì sao tắt cascade delete? Đáp ý chính: không tự xóa dây chuyền thiết bị khi xóa loại; phải xử lý phụ thuộc phù hợp.

16. AsNoTracking có tác dụng gì? Đáp ý chính: đọc không cần theo dõi thay đổi. Trong phép chiếu DTO của bài, kết quả cũng không phải entity để sửa trực tiếp.

17. Tại sao dùng Find khi sửa? Đáp ý chính: tìm theo PK, lấy entity được theo dõi, kiểm tra null rồi gán thuộc tính không khóa.

18. using có xóa database không? Đáp ý chính: không; Dispose context/tài nguyên, dữ liệu đã commit vẫn tồn tại.

Nhóm C — Nghiệp vụ

19. Mã trùng được chặn ở đâu? Đáp ý chính: BLL Trim rồi TonTai; PK SQL vẫn chặn race.

20. Required có chặn số lượng âm không? Đáp ý chính: không; điều kiện >0 trong KiemTra mới chặn nghiệp vụ đó.

21. Vì sao vẫn kiểm tra độ dài trong BLL khi TextBox có MaxLength? Đáp ý chính: GUI chỉ là một đường nhập; service phải tự bảo vệ đầu vào.

22. TryParse thay Parse để làm gì? Đáp ý chính: sai định dạng trả false, BLL đưa thông báo nghiệp vụ thay vì FormatException trực tiếp.

23. Giá 15000000,50 có hợp lệ không? Đáp ý chính: có với culture vi-VN và các trường khác hợp lệ; dấu phẩy là thập phân.

24. Vì sao không dùng double cho giá? Đáp ý chính: số thực nhị phân có sai số; decimal phù hợp tiền và khớp SQL decimal.

25. Vì sao so NgayMua.Date? Đáp ý chính: ràng buộc theo ngày, bỏ phần giờ; ngày hôm nay vẫn hợp lệ.

26. Tại sao cấm đổi MaTB khi sửa? Đáp ý chính: giữ định danh, tránh biến sửa thành thiết bị khác; khóa GUI, kiểm tra BLL và DAL không gán PK.

27. Hai người cùng nhập mã mới thì sao? Đáp ý chính: kiểm tra trước vẫn có race; database PK là chốt cuối, GUI phải xử lý lỗi lưu.

28. Hai người sửa cùng một dòng thì sao? Đáp ý chính: bản này chưa có rowversion; có nguy cơ lần lưu sau ghi đè. Hướng cải tiến là optimistic concurrency và thông báo xung đột.

Nhóm D — GUI và async

29. DisplayMember khác ValueMember? Đáp ý chính: tên hiện ra và mã dùng để lưu/chọn FK.

30. Vì sao cần dangGanDuLieu? Đáp ý chính: bind/reset bằng code cũng phát sinh SelectionChanged; cờ giúp bỏ qua sự kiện ngoài ý định người dùng.

31. Vì sao nút Nhập tắt khi chọn dòng? Đáp ý chính: đang ở chế độ sửa/xóa, tránh thêm nhầm mã đã có.

32. Mới có xóa dữ liệu SQL không? Đáp ý chính: không; chỉ reset nhập liệu/lựa chọn trên GUI.

33. Vì sao dùng Shown thay vì chặn Form Load? Đáp ý chính: cho cửa sổ hiện trước khi công việc SQL có thể chờ lâu; sau nạp vẫn đưa về trạng thái đúng đề.

34. Task.Run có tự làm EF thread-safe không? Đáp ý chính: không. Không chia sẻ context giữa thread; bài tạo context trong mỗi repository.

35. Có cập nhật TextBox trong Task.Run không? Đáp ý chính: không; chụp input trước, await xong trở lại UI thread rồi bind.

36. Vì sao lỗi lưu và lỗi tải lại phải tách? Đáp ý chính: nếu đã lưu thì không yêu cầu nhập lặp; daLuu phân biệt hai giai đoạn.

37. Làm sao thoát chỉ hỏi một lần? Đáp ý chính: nút gọi Close; duy nhất FormClosing xác nhận; No đặt e.Cancel=true.

Nhóm E — Mở rộng và kiểm thử

38. Thêm Thiết bị mạng có cần Migration? Đáp ý chính: không nếu chỉ thêm dòng loại; có nếu thay schema.

39. Test giả chứng minh điều gì? Đáp ý chính: nhánh nghiệp vụ theo hợp đồng, không chứng minh SQL/WinForms/migration thật chạy đúng.

40. Nếu cô hỏi phần chưa làm? Đáp ý chính: nói đúng giới hạn, nêu nơi cần sửa và cách kiểm thử; không nhận là có sẵn tính năng chưa triển khai.

18. Vấn đáp giả lập và tờ ôn nhanh

Kịch bản 1 — “Em chỉ một lần Nhập chạy qua đâu?”

Mở FrmThietBi.LuuAsync, chỉ DocDuLieu và lời gọi service.Nhap. Mở ThietBiService.Nhap/KiemTra. Mở DAL.Them, chỉ Add và SaveChanges. Quay về ThucHienGhiAsync, chỉ LayDanhSach và GanDanhSach. Demo TB006 rồi thử mã trùng.

Kịch bản 2 — “Không cho sửa mã, em làm ở đâu?”

Chỉ txtMaTB.ReadOnly trong ChonDong, so maGoc trong BLL.Sua và các phép gán thuộc tính của DAL.Sua không có MaTB. Nói rõ kiểm soát GUI không thay thế bảo vệ nghiệp vụ.

Kịch bản 3 — “Thêm MANG nhưng Wi-Fi vẫn là máy tính?”

Chỉ Seed bổ sung danh mục MANG; chỉ TB003 vẫn còn MaLoai=MAYTINH trong mẫu của commit này. Nói rõ database đã có không bị Seed ghi đè. Demo sửa loại bằng UI nếu cần, không xóa cả database.

Kịch bản 4 — “Em thêm cột phòng học ngay bây giờ?”

Trình bày ảnh hưởng từ entity → DTO → BLL → DAL → GUI → Migration; hỏi rõ cột bắt buộc hay tùy chọn và giá trị cho dữ liệu cũ. Chưa nên gõ migration khi chưa thống nhất ràng buộc.

Ôn 15 phút trước khi thi

- Nói 1 phút bài toán và ba tầng.

- Vẽ 2 bảng, PK/FK, thuộc đúng độ dài 10/100/20/150 và decimal(18,2).

- Truy vết Nhập, Sửa, Xóa; phân biệt Add với SaveChanges.

- Nhớ năm cờ/giá trị dễ hỏi: maDangChon, sanSang, dangXuLy, dangGanDuLieu, daLuu.

- Phân biệt Migration với Seed và schema với dữ liệu.

- Demo một đầu vào hợp lệ, trùng mã, ngày mai, số lượng 0 và hủy thoát.

- Chuẩn bị đường dẫn file; không cần nhớ số dòng vì có thể thay đổi.

Thang tự đánh giá

Mỗi năng lực chấm 0: chưa biết; 1: nói được nhưng chưa chỉ code; 2: nói được và chỉ đúng; 3: nói/chỉ code/demo/suy luận biến thể. Chấm sáu năng lực: kiến trúc, dữ liệu, EF/Migration, validation, CRUD, trạng thái/lỗi. Ưu tiên luyện những mục dưới 2 điểm. Điểm này là tự đánh giá, không phải dự báo điểm thi.

Khi gặp câu mới: xác định cô đang hỏi UI, nghiệp vụ, dữ liệu hay môi trường; nhắc lại yêu cầu; nêu đường xử lý; mở đúng hàm; đưa ví dụ hợp lệ và một phản ví dụ. Nếu chưa chắc, nói phần biết và cách kiểm chứng thay vì đoán.

Phụ lục. Chỉ mục hàm trong bản source được đối chiếu

File	Hàm cần tìm	Dòng ở commit đối chiếu
QuanLyThietBi.GUI/Program.cs	Main	12
QuanLyThietBi.GUI/FrmThietBi.cs	TaoGiaoDien	53
QuanLyThietBi.GUI/FrmThietBi.cs	CapNhatNut	164
QuanLyThietBi.GUI/FrmThietBi.cs	NapDuLieuAsync	184
QuanLyThietBi.GUI/FrmThietBi.cs	GanDanhSach	222
QuanLyThietBi.GUI/FrmThietBi.cs	LamMoi	233
QuanLyThietBi.GUI/FrmThietBi.cs	ChonDong	246
QuanLyThietBi.GUI/FrmThietBi.cs	DocDuLieu	256
QuanLyThietBi.GUI/FrmThietBi.cs	LuuAsync	261
QuanLyThietBi.GUI/FrmThietBi.cs	XoaAsync	268
QuanLyThietBi.GUI/FrmThietBi.cs	ThucHienGhiAsync	276
QuanLyThietBi.GUI/FrmThietBi.cs	XacNhanThoat	323
QuanLyThietBi.BLL/ThietBiService.cs	KiemTra	25
QuanLyThietBi.BLL/ThietBiService.cs	Nhap	47
QuanLyThietBi.BLL/ThietBiService.cs	Sua	53
QuanLyThietBi.BLL/ThietBiService.cs	Xoa	61
QuanLyThietBi.DAL/Repositories.cs	LayTatCa	10, 50
QuanLyThietBi.DAL/Repositories.cs	TonTai	19, 54

File	Hàm cần tìm	Dòng ở commit đối chiếu
QuanLyThietBi.DAL/Repositories.cs	Them	23
QuanLyThietBi.DAL/Repositories.cs	Sua	27
QuanLyThietBi.DAL/Repositories.cs	Xoa	38
QuanLyThietBi.DAL/ThietBiContext.cs	OnModelCreating	17
QuanLyThietBi.DAL/Migrations/Configuration.cs	Seed	17
QuanLyThietBi.DAL/DatabaseBootstrapper.cs	Initialize	8